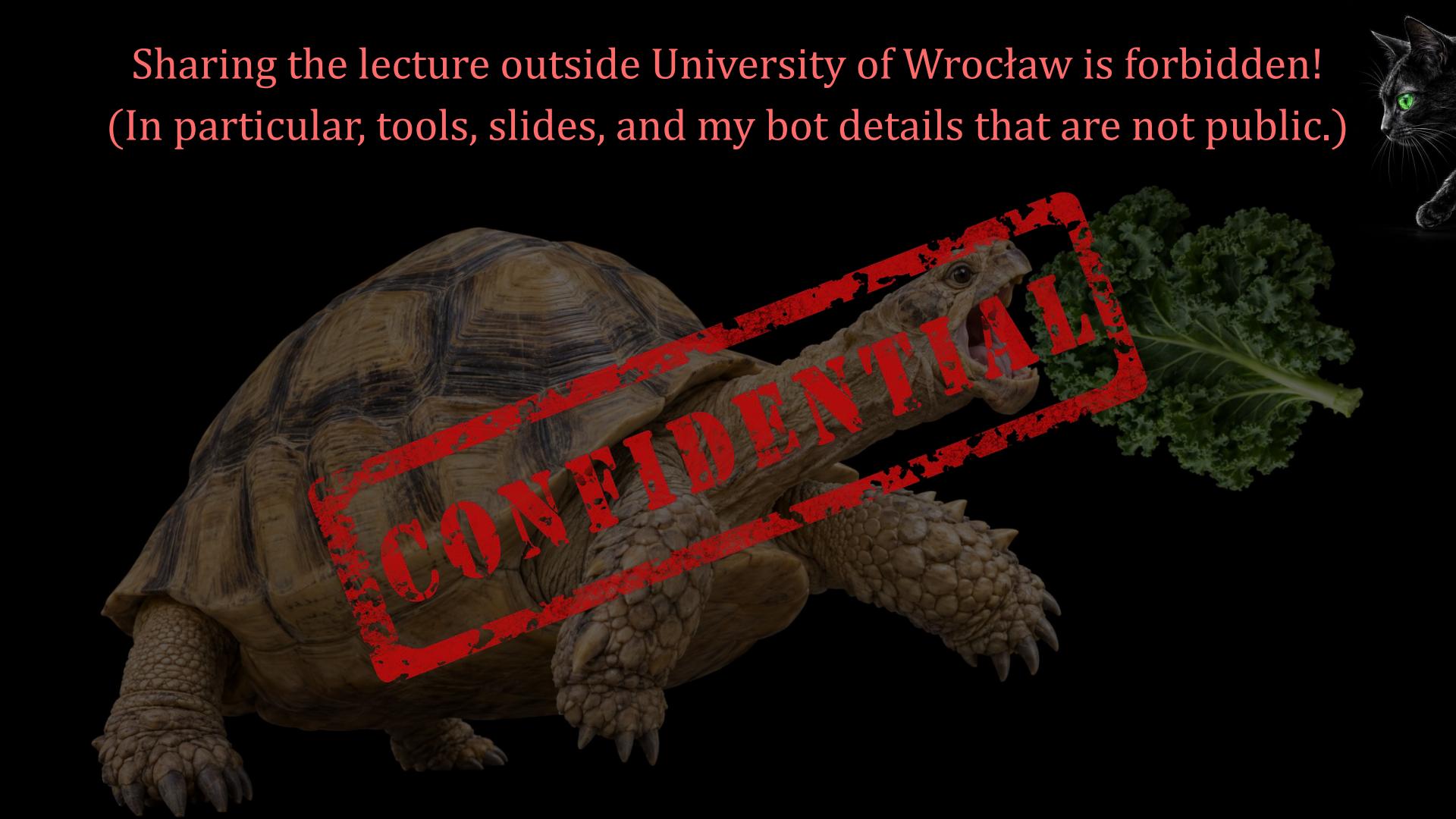
Mastering Game Bots

Lecture 2



- Local benchmarker setup
- Game state, actions
- Evaluation function

Sharing the lecture outside University of Wrocław is forbidden!
(In particular, tools, slides, and my bot details that are not public.)



Local benchmarker (mine) [demonstration]

1. Prepare the benchmarker project.
   a. Download the source code of the game referee. The link is usually at the end of the game description.
   b. Add the benchmarker code to this project (available at SKOS). Pay attention to put the source code inside the right package (directory structure).
2. Set desired time limits.
   a. Decompile GameManager.class and it to the project as a GameManager.java.
   b. Remove the **if**s restricting time limits that can be set:
      in methods setMaxTime, setTurnMaxTime, and addTurnTime.
   c. Set larger time limits in Referee.java, e.g., 5s and 2s.
3. Fix the referee for repetitive usage in the same process (sometimes required).
   a. Investigate the referee code for changed static variables (most often a global id counter).
      They must be reset before every game simulation in RefereeCaller.java.
   b. Investigate the referee code for any uncontrolled nondeterminism.
      E.g., replace HashSet with LinkedHashSet and HashMap with LinkedHashMap,
      but only if the order matters.
4. Add agents' executables to agents.
5. Run Benchmarker directly in IntelliJ or in terminal:

```
mvn clean && mvn install && mvn dependency:copy-dependencies
java -cp target/NAME-1.0-SNAPSHOT.jar:target/dependency/*  benchmarker.Benchmarker
```

(There are some scripts too.)

Other testing tools (if you want)

- Brutal tester by dreignier (direct testing, like my Benchmarker).
  - The oldest local testing tool.

- Psyleague by Psyho – a generic ranking-based framework

```
> info msz_22_exe
Pos Name        Score      % Wins Losses Draws
--- ---------- ------ ------ ---- ------ -----
 1  Player57_6 26.142 94.672  115    6       1
 2  Player57_1 26.076 87.619   92   13       0
 3  Player57_5 26.007 86.614  110   17       0
 4  Player57_3 26.001 84.694   83   15       0
 5  Player57_2 25.941 89.062  114   14       0
```

- CGArena by aangairbender
  - The most advanced and recently developed.

One common disadvantage

- They all start a new referee (Java) process for each play.
- About ~20% slower than the Benchmarker
  (heavy depends on the system and plays length; I measured it long time ago, though).

Overview of main bot ingredients

1. Rule-based action selection
   - `if (canAttack()) doAttack(); else`
     `if (canGather()) doGather(); else`
     …
   - Generally the weakest, but the fastest and the simplest.
   - Useful only as initial choices or in deeper heavy simulations.

2. Simulation
   - Minmax family, Beam search family, Monte-Carlo Tree Search (MCTS), Hill Climbing (HC), etc.
   - Requires evaluation of game states.
   - Solver for endings.

3. Engine (computing moves and game states)
   - Crucial for algorithmic search.
   - Must (or should) be exact and efficient.

4. Evaluation function (`GameState -> int,int64` or `float,double`).
   - Lexicographical: `(scoreEval, resourceEval, positionEval, …)`
   - Mix components (better, but be careful of unintended overlaps):
     `SCORE_WEIGHT*scoreEval + RES_WEIGHT*resourceEval + POS_WEIGHT*positionEval + … …`

Game state

- Define State and Actions, and a function to apply the actions.
- They must be small and of a fixed size.
- Pay attention to data types and padding.
  - Not necessarily minimal: int and uint are sometimes faster than smaller types.
- Hint: leave optimization for later (e.g., bit encoding, auxiliary data).

- Typical template (for simultaneous actions):

```
struct State {
uint score[2];
…
};

using Actions = std::array<Action,MAX_ACTIONS>;

void State::apply(const Actions &a1,const Actions &a2) {
  // Modify the state
}
```

Game state example – SnakeByte

```
using Actions = std::array<Dir,MAX_SNAKES>;
using JointActions = std::array<Actions,2>;

uint glCurrentTurn;

struct BSnake {
uint len;
uint headX,headY;
uint tailX,tailY;
u64 dirs[3];// From head to tail, directions toward the head
Mask body;

};

struct BState {
Mask foods,wallsAndSnakes;
int losses;
BSnake snakes[2][MAX_SNAKES];

};
```



Experiment with LLMs for SnakeByte

- GPT-5.5 Terra: Implemented a correct bot, but made terrible efficiency decisions.

- Claude Sonnet 5.5: Implemented a bot that timeouts sometimes, but technically seems better.

```cpp
struct Snake {
    int id;
    int owner;
    vector<Pos> body;
    int dir = 0;
    bool alive() const { return !body.empty(); }
};

struct World {
    vector<Snake> snakes;
    vector<Pos> food;
};

class Bot {
    static constexpr int DX[4] = {0, 1, 0, -1};
    static constexpr int DY[4] = {-1, 0, 1, 0};
    static constexpr const char* NAME[4] = {"UP", "RIGHT", "DOWN", "LEFT"};

    int me, width, height, botsPerPlayer;
    vector<string> terrain;
    vector<int> myIds, enemyIds;
    unordered_set<int> myIdSet;
    vector<int> lastDir;
    chrono::steady_clock::time_point deadline;
```

```cpp
static const int MAXS = 16, MAXL = 256, MAXC = 45 * 30, D = 12;
static const int DX[4] = {0, 0, -1, 1}, DY[4] = {-1, 1, 0, 0};
static const char* DN[4] = {"UP", "DOWN", "LEFT", "RIGHT"};

int W, H;
bool wall[MAXC];
int field[MAXC];

struct Snake {
    int n = 0;
    bool alive = false, mine = false;
    int id = 0;
    short x[MAXL], y[MAXL];
};
struct State {
    int ns = 0;
    Snake s[MAXS];
    bool food[MAXC];
    int foodCnt = 0;
};
```

Typical evaluation ingredients (examples)

1. Score.
   - Raw score (`my_score - opponent_score`).
   - Bonus for having a greater score than the opponent.
2. Territory.
   - Raw distances to something (e.g., `max - distance` or `1.0/distance` bonus; distance can be squared or raised to a power).
   - Voronoi (the player who is closer gets the bonus).
   - Bonus/penalty: a stronger agent chases a weaker agent.
3. Resources.
   - Raw quantities in inventory.
   - Current gain (income).
   - Missing resources for what we need. Penalty for gathering time.
   - Bonus for immediate gather (in the next turn).
4. Agents / Structure.
   - Number of agents.
   - Health.
   - Mobility.
   - Ammunition, cooldown.
   - Penalty for vulnerabilities.
5. Meta (adjust the weights, turn on/off features).
   - Based on global meta decisions (e.g., what to build next, where should we go).
   - Based on game phase (e.g., turn number, ungathered resources remaining).

Hints

1. Think about the game mechanism.
   - Turns are the currency.
2. Start from the most important features.
   - Usually score, agents, resources.
3. Pay attention to computation cost.
4. Test ***everything*** .
   - Start from smaller weights to see if a feature works at all.
5. Implement conditionally.
   - A reasonable feature not benefiting now may turn out to be useful later.
   - `if constexpr(FEATURE_X) {...}`

SnakeByte – examples of evaluation ingredients



SnakeByte – examples of evaluation ingredients

BFS distance to the closest food – unrestricted snake length and hang point



SnakeByte – examples of evaluation ingredients

BFS distance to the closest food – snake length 3 and hang point 0 (head)



SnakeByte – examples of evaluation ingredients

BFS distance to the closest food – snake length 6 and hang point 5 (tail)



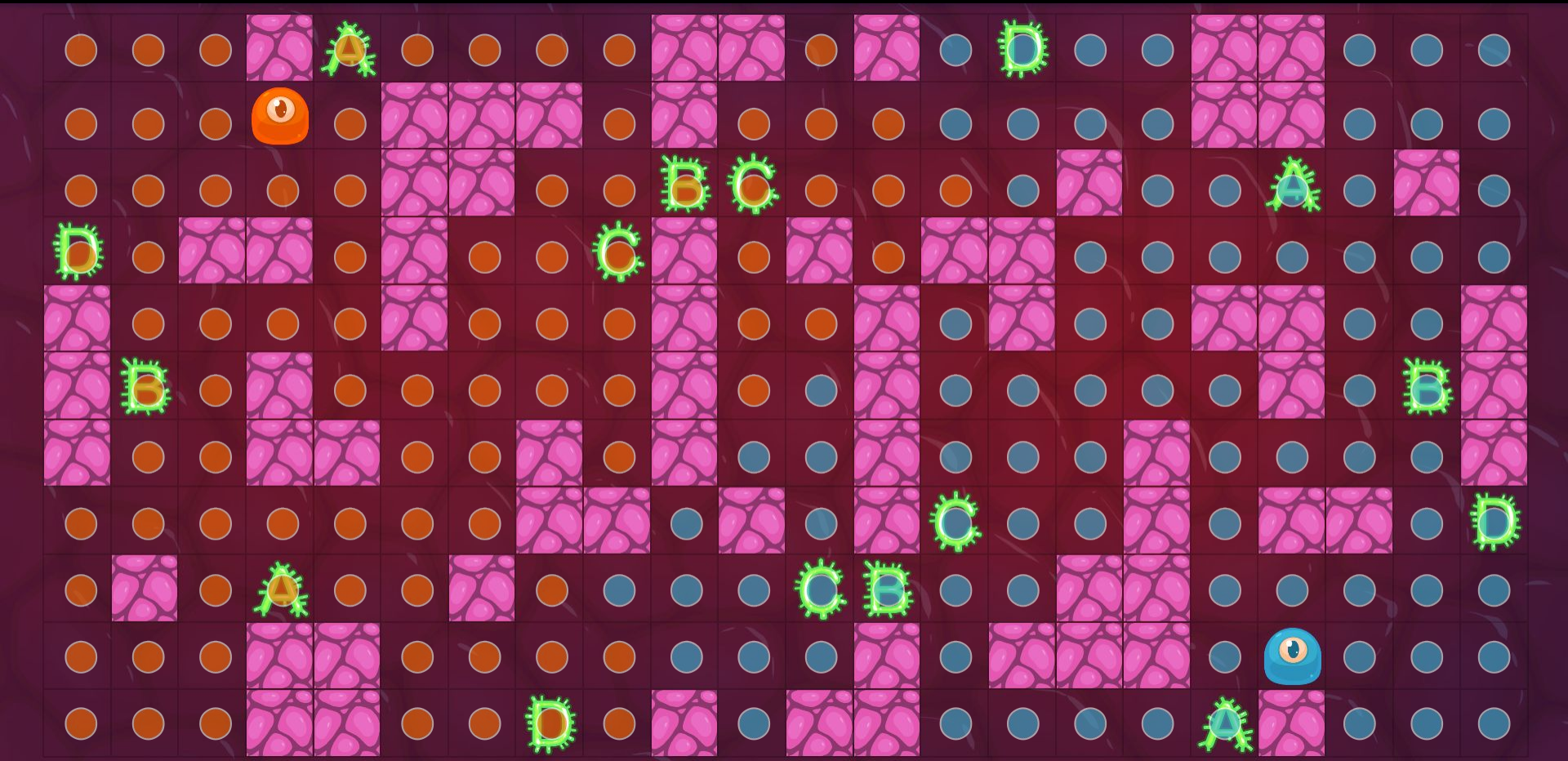
SnakeByte – examples of evaluation ingredients

- I preprocess snake lengths from 3 to 9, and the unrestricted case.
- It takes about 2 ms for this board.
- I tried to extend it for falling snakes, but it was too expensive...

Some other eval ingredients:

- Distance to every food separately (unrestricted case).
- Hang points
- Mobility
- Distance to opponent (me longer: advantage; me shorter: penalty)
- Penalty for small space in the current map component
- Penalty for being outside the board
- Penalty for up-vertical snake 🙂

Cellularena – Voronoi example (turn 0)



Cellularena – Voronoi example (turn 10)

